Sorbonne Université

Academic year : 2025-2026

Y2 Master of Engineering for Health (MSR)

MU5EEH15 Social Robotics : CarRacing-v2 environment

Project report

Presented on November 29th, 2025

by

Jacques GUÉRIN et William WU

Academic supervisors: Mohamed CHETOUANI, Louis SIMON, Silvia TULLI

Table des matières

Introduction

The field of Social Robotics is fundamentally concerned with developing agents that can learn from and adapt to humans in an intuitive and interactive manner. A core challenge in this domain is enabling robots to acquire complex skills without the need for meticulously engineered reward functions, which are often difficult to specify and can fail to capture the nuances of human intent. Instead, learning from demonstrations (LfD) provides a powerful alternative, allowing an agent to infer a policy directly from expert behavior.

In this project, we address the challenge of scaling interactive learning to complex, high-dimensional environments. While foundational algorithms like Behavioral Cloning offer a simple starting point, they often suffer from cascading errors due to a lack of feedback and an inability to recover from mistakes. More advanced, interactive frameworks like TAMER integrate human feedback but can become inefficient in environments with continuous state and action spaces, where providing precise, timely feedback is challenging.

To tackle these limitations, we propose a framework centered on **Imitation Learning (IL)**. Our work explores the transition from simple Behavioral Cloning to more robust, adversarial imitation methods. After a theoretical and practical exploration of various algorithms, we selected **Generative Adversarial Imitation Learning (GAIL)** [1]. GAIL combines the power of Generative Adversarial Networks (GANs) with imitation learning, enabling an agent to learn a policy that is indistinguishable from expert demonstrations by training a discriminator to differentiate between expert and agent trajectories.

We test our approach in the **CarRacing-v2** environment from Gymnasium [2]. This environment presents a significant challenge with its high-dimensional, continuous action space (steering, acceleration, braking) and complex, pixel-based state representation (cf. Fig 1). It serves as an excellent testbed for our goals, requiring a model that can learn smooth, expert-level control policies from a limited set of demonstrations.

FIGURE 1 – CarRacing-v2 environment's graphical interface

Category	**Command**	**Keyboard Key**
Steering	Left	[Q]
	Right	[D]
Throttle	Accelerate	[Z]
	Brake/Stop	[S]

TABLE 1 – Control mapping for CarRacing-v2 environment

The primary objective of this project is to implement and evaluate the GAIL algorithm within the CarRacing-v2 environment. We aim to demonstrate that GAIL can effectively learn a robust racing policy from expert data, overcoming the limitations of simpler imitation learning methods and providing a foundation for future integration of real-time human feedback. This report details our background research, algorithmic implementation, experimental results, and a discussion on the insights of our work.

1

Background and Theoretical Foundations

The development of autonomous agents capable of learning complex behaviors from human teachers is a fundamental principle of social robotics. A primary pathway to this goal is imitation learning, which allows an agent to acquire a policy by observing expert demonstrations, thereby bypassing the need for a manually engineered reward function. This section outlines the theoretical journey from simple interactive learning methods to the more advanced adversarial approach that forms the basis of our project.

1.1 From Behavioral Cloning to Adversarial Imitation Learning

A straightforward approach to imitation learning is Behavioral Cloning (BC) [3], which frames the problem as supervised learning. In BC, the agent learns a direct mapping from states to actions using the expert's state-action pairs as a labeled dataset. While effective in limited scenarios, BC suffers from fundamental limitations that hinder its application to complex, real-world tasks.

The most critical issue is compounding errors. Since the trained policy is not perfect, it will inevitably make small mistakes, leading it to states that were not present in the expert's original distribution. In these unfamiliar states, the BC policy is likely to make another error, further deviating from the expert's trajectory. This cascade of errors, known as distribution shift or the « covariate shift » problem, often causes the agent to fail catastrophically in long-horizon tasks, even if it performed perfectly on the training data.

This core weakness of BC motivates the need for more robust algorithms that interact with the environment during learning. Methods like DAgger (Dataset Aggregation) address this by iteratively collecting data from the agent's own policy and having the expert relabel it, thus learning how to recover from mistakes. However, DAgger requires ongoing, active involvement from the expert, which can be impractical and difficult to put into practice.

An alternative paradigm is Inverse Reinforcement Learning (IRL), which aims to infer the underlying reward function that the expert is optimizing, rather than just copying their actions. While powerful, traditional IRL methods are often computationally prohibitive for high-dimensional environments. The need for a method that is both robust to distribution shift and scalable to complex problems led to the development of Adversarial Imitation Learning, a framework that combines the stability of environment interaction with the power of IRL.

1.2 Generative Adversarial Imitation Learning (GAIL)

Generative Adversarial Imitation Learning (GAIL) [1] emerged as a groundbreaking framework that addresses the limitations of Behavioral Cloning by combining the scalability of deep reinforcement learning with the theoretical foundations of Inverse Reinforcement Learning. Inspired by Generative Adversarial Networks (GANs) [4], GAIL formulates imitation learning as a distribution-matching problem between the agent's and the expert's state-action trajectories.

Core Idea : Imitation as Distribution Matching

The fundamental objective of GAIL is not simply to copy individual actions, but to match the *entire distribution* of the expert's behavior. The algorithm aims to produce an agent whose policy, π, generates trajectories $\tau = (s_0, a_0, s_1, a_1, ...)$ that are statistically indistinguishable from the expert's demonstrations. This high-level goal makes the learned policy much more robust to the distribution shift that plagues BC.

The Adversarial Game

This distribution matching is achieved through an adversarial game between two neural networks :

– **The discriminator (D) :** This network acts as a classifier. Its goal is to correctly distinguish between state-action pairs (s, a) coming from the expert's demonstrations and those generated by the agent's policy π. It is trained to output a value close to 1 for expert data and close to 0 for agent data. The loss function for the discriminator is typically a binary cross-entropy loss.

– **The generator (Policy π) :** The policy π is the agent we are training. Its goal is to "fool" the discriminator by generating state-action pairs that are so expert-like that the discriminator assigns them a high value (close to 1). The agent is trained using standard reinforcement learning, where the **reward at each timestep is provided by the discriminator**. Specifically, the reward $r(s, a)$ is often taken as $-\log(1 - D(s, a))$, meaning the agent gets a high reward when the discriminator is fooled (i.e., when $D(s, a)$ is high).

This creates a continuous feedback loop : the discriminator gets better at telling the agent and expert apart, the policy, in its effort to maximize the reward from the discriminator, is forced to generate behavior that becomes progressively more similar to the expert's to fool the increasingly powerful discriminator.

This adversarial training process can be visualized as a minimax optimization, where the policy and discriminator are jointly optimized according to the following objective [1] :

$$\min_{\pi} \max_{D} \mathbb{E}_{\pi}[\log D(s,a)] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))] - \lambda H(\pi)$$

where π_E is the expert's policy and $H(\pi)$ is an entropy regularization term encouraged exploration.

1.3 Algorithm Exploration

The selection of an appropriate imitation learning algorithm is important for balancing performance, computational efficiency, and implementation complexity. Before settling on GAIL, we conducted a theoretical exploration of several prominent algorithms to assess their suitability for the *CarRacing-v2* environment. Our exploration focused on methods that address the limitations of Behavioral Cloning.

Adversarial Inverse Reinforcement Learning (AIRL [5] : As a successor to GAIL, AIRL aims to not only imitate the expert but also recover a reward function that is robust to changes in dynamics (invariant to dynamics). While this is a theoretically appealing property, its implementation is significantly more complex than GAIL, requiring a separate reward network and careful balancing of losses. Given the project's scope and timeline, we prioritized a more stable and well-documented baseline.

DAgger [6] : The Dataset Aggregation algorithm directly addresses the distribution shift problem by iteratively querying the expert for labels on states visited by the agent's current policy. This makes it very data-efficient and robust. However, its core requirement for an *interactive, online expert* to provide corrective labels throughout training was a major limiting factor. For *CarRacing-v2*, this would necessitate a human expert continuously playing the game to correct the agent's mistakes, which is impractical for large-scale training.

Soft Q Imitation Learning (SQIL) [7] : SQIL is a simpler algorithm with a constant reward for expert-like states and zero reward otherwise. It can be implemented with standard off-the-shelf RL algorithms. However, preliminary research and existing literature suggested that while SQIL can be effective, it often underperforms compared to adversarial methods like GAIL in environments with complex dynamics, as it lacks the sophisticated distribution-matching objective.

A practical consideration that further reinforced our choice was the seamless integration of GAIL with **Proximal Policy Optimization (PPO)** [8] through the Stable-Baselines3 library [9]. PPO has demonstrated exceptional performance in continuous control tasks like *CarRacing-v2*, providing stable and sample-efficient policy optimization. The proven success of PPO in this specific environment, combined with its robust implementation in Stable-Baselines3, gave us confidence that the policy optimization component of GAIL would be reliable and effective. This practical advantage, supported by documented results in similar continuous control domains, made GAIL with PPO as the underlying RL algorithm a technically sound choice.

Our final choice, **Generative Adversarial Imitation Learning (GAIL)**, emerged as the most suitable candidate for this project for several key reasons :

- **Scalability :** GAIL is designed to handle high-dimensional, continuous state and action spaces through the use of deep neural networks, making it a natural fit for the pixel-based inputs and continuous control of *CarRacing-v2*.
- **Offline Demonstrations :** Unlike DAgger, GAIL can learn directly from a fixed dataset of pre-recorded expert trajectories, which is logistically far simpler than coordinating an interactive expert.
- **Proven Performance :** GAIL has a strong track record in the literature for learning robust policies in complex environments, from robotic locomotion to autonomous driving tasks.
- **Conceptual Foundation :** The adversarial framework provides a principled and powerful approach to distribution matching, offering a significant theoretical and practical upgrade over Behavioral Cloning without the extreme complexity of some other IRL methods.
- **Practical Integration :** The availability of well-tested implementations combining GAIL with PPO in libraries like Stable-Baselines3 ensures training stability and reproducibility.

Therefore, GAIL presents the optimal balance of theoretical robustness, practical implementability, and proven performance for mastering the *CarRacing-v2* environment.

Having established the theoretical rationale for selecting Generative Adversarial Imitation Learning, we now detail its practical application. This section describes the experimental setup, encompassing the learning environment, data collection process, and the specific implementation of the GAIL algorithm.

2

Implementation and difficulties

2.1 Environment : CarRacing-v2

This section outlines the technical pipeline developed to train our autonomous racing agent. We describe the simulation environment, the procedures for acquiring expert knowledge, the architectural choices for our models, and the training methodology that integrates these components. The **CarRacing-v2** environment presents a top-down view of a racing track, posing a significant imitation learning problem.

Environment Characteristics :

— **State Space :** The primary observation is a high-dimensional RGB image of the game screen, with a resolution of 96x96 pixels. For efficient processing by our neural networks, we convert this to the channel-first format (3, 96, 96). This pixel-based input requires the policy network to learn relevant features, such as track borders and the car's orientation, directly from the raw visual data.

— **Action Space :** The agent controls the car using a **discrete action space** with five predefined driving maneuvers that cover the essential control spectrum. Each discrete action maps to a continuous control vector [steering, acceleration, braking] that interfaces with the CarRacing environment's native continuous action space. This discretization provides training stability and reduces the complexity of the policy learning problem.

— **Objective :** The agent's goal is to navigate the track as quickly as possible. While the default environment provides a small negative reward for each frame and a large positive reward upon completing a lap, our GAIL implementation primarily uses the adversarial reward signal from the discriminator, with optional environment reward blending for stability.

Observation Preprocessing :

The pixel were processed as follows to ensure compatibility with our PyTorch-based networks :

1. **Channel Reordering :** The RGB input in HWC format (96, 96, 3) was converted to CHW format (3, 96, 96) to align with PyTorch's convolutional layer requirements.
2. **Data Type Conversion :** Pixel values were converted to float32 and normalized as needed during network forward passes.
3. **Direct Pixel Input :** Raw RGB pixels were fed directly to the networks, allowing convolutional layers to learn feature representations automatically without extensive preprocessing.

This streamlined approach minimizes computational overhead while leveraging the representational capacity of modern neural architectures to extract relevant features from visual inputs.

2.1.1 GAIL Implementation

The motivation behind integrating GAIL into the CarRacing project is to exploit expert demonstrations to guide a reinforcement learning agent toward robust and human-like driving behavior. Given the environment's complexity with its high-dimensional visual state space and need for precise control, GAIL provides a principled framework to learn a policy that reproduces expert behavior while maintaining training stability.

Overall Architecture

Our GAIL implementation consists of two main components working in tandem :

— **Policy Network (PPO Agent) :** A discrete action policy based on the Proximal Policy Optimization algorithm that generates driving behaviors.

— **Discriminator Network :** A binary classifier that distinguishes between expert and policy-generated state-action pairs, providing the adversarial training signal.

Discrete Action Space Strategy

We implemented a discretized action space with five carefully chosen driving maneuvers that cover the essential control spectrum. This discrete approach provides several advantages : enhanced training stability by reducing the complexity of the policy learning problem, improved sample efficiency by focusing learning on meaningful control patterns, and increased robustness by avoiding the challenges of fine-grained continuous control in high-dimensional spaces.

Adversarial Training Mechanism

The interaction between PPO policy and the discriminator forms an iterative adversarial loop :

1. **Trajectory Collection :** The current policy interacts with the CarRacing environment, collecting sequences of state-action pairs (CHW format and discrete action indices).
2. **Discriminator Training :** The discriminator is updated using both expert demonstrations and policy-generated samples with label smoothing for stability.
3. **Reward Computation :** The discriminator's output is transformed into a training signal using stable reward computation with clipping and normalization to prevent gradient explosion.
4. **Policy Optimization :** The PPO agent updates its policy using the adversarial rewards, with optional environment reward blending for stability, typically using a 70% GAIL to 30% environment reward ratio.

Enhanced Training Strategies

We wanted some improvements over basic GAIL for our implementation. For enhanced training stability, we employed gradient clipping in discriminator updates, label smoothing for discriminator training, reward normalization and clipping, and conservative policy updates limited to 1-2 epochs per iteration.

To ensure robust learning behavior, we implemented behavioral safeguards including action diversity monitoring to prevent policy collapse, early stopping mechanisms for degenerate behaviors, and progressive training difficulty. Furthermore, our BC-GAIL integration strategy incorporates warm-starting from Behavioral Cloning models, conservative fine-tuning to preserve prior knowledge, and balanced reward blending between imitation and environment feedback.

Training Pipeline : the training process follows this workflow :

1. **Initialization :** Load expert demonstrations and can initialize with a BC-pretrained policy
2. **Rollout Collection :** Generate policy trajectories using the current policy
3. **Discriminator Update :** Improve discrimination between expert and policy data
4. **Reward Computation :** Generate GAIL rewards from discriminator outputs
5. **Policy Update :** Optimize policy using PPO with adversarial rewards
6. **Evaluation & Saving :** Monitor performance and save improved models

Despite extensive experimentation with our enhanced GAIL framework, we were unable to produce a successful driving policy. The agent consistently converged to degenerate behaviors, typically remaining stationary regardless of training duration or parameter adjustments. This demonstrates the practical challenges of adversarial imitation learning in complex environments like autonomous racing.

2.2 Problems Encountered

During the implementation and training of our GAIL framework, we encountered several significant challenges that impacted the project's outcomes and revealed limitations in our approach.

Training instability and convergence issues : The adversarial nature of GAIL training proved highly unstable in practice. Despite implementing numerous stability enhancements including gradient clipping, label smoothing, and conservative policy updates, the training process frequently diverged or converged to degenerate behaviors. The discriminator often became either too powerful too quickly, providing no useful learning signal to the policy, or too weak to distinguish between expert and policy behaviors. This delicate balance between discriminator and policy training proved difficult to maintain, with most training runs resulting in policies that either remained stationary or exhibited chaotic, non-driving behaviors.

Practical Implementation Challenges : Our implementation revealed several practical limitations. The computational demands of simultaneously training both discriminator and policy networks were substantial, requiring extensive GPU resources for what ultimately yielded poor results. The discrete action space, while simplifying the control problem, may have been too restrictive to capture the nuanced steering and acceleration patterns needed for competent driving. Additionally, the frame-by-frame processing without temporal stacking likely limited the policy's ability to infer velocity and motion, essential elements for coherent driving behavior.

Failure to Transfer BC Knowledge : A particularly disappointing outcome was the failure of our BC-GAIL integration approach. Despite warm-starting with Behavioral Cloning models that showed promising initial performance, the GAIL fine-tuning process consistently degraded rather than improved the policy. The conservative update strategy intended to preserve BC knowledge proved insufficient, as the adversarial training dynamics overwhelmed the pre-learned behaviors. This resulted in final policies that performed worse than the initial BC models, with average rewards typically remaining negative throughout training.

Algorithmic Limitations in Complex Environments : The fundamental GAIL approach demonstrated limitations when applied to the CarRacing environment. The high-dimensional visual state space, combined with the precise temporal coordination required for driving, appeared to exceed what our GAIL implementation could effectively learn. The discriminator struggled to provide meaningful reward signals beyond superficial behavioral matching, failing to guide the policy toward coherent, goal-directed behavior. This was evident in the consistent failure to achieve positive rewards, with most training runs never surpassing the critical threshold where the car learns to maintain forward motion and track following.

Despite extensive experimentation with different architectures, reward formulations, and training strategies, we were unable to produce a GAIL policy that demonstrated competent driving behavior. The combination of adversarial training instability, computational constraints, and environmental complexity proved insurmountable within the scope of this project.

3

Change of strategy

Seeing that our efforts to implement GAIL correctly in CarRacing-v2 were in vain, we decided to refocus the project on a simpler imitation learning approach : Behavioral Cloning. We kept the PPO agent and the human expert demonstrations and used BC to enhance the agent. We also used BC paired with the expert's demonstrations to create a model based on pure imitation learning. Therefore, we could compare and analyze three models for the project :

— **PPO baseline** : a standard PPO agent trained from scratch using the environment reward.
— **BC-enhanced PPO** : the PPO agent initialized and regularized by Behavioral Cloning.
— **BC-human imitation** : a purely supervised policy trained to predict expert actions from observations.

The following section describes Behavioral Cloning in depth, details our implementation choices, presents the empirical results and comparisons between the three approaches, and provides a conclusion for the project.

3.1 Behavioral Cloning

3.1.1 Problem formulation

Behavioral Cloning frames imitation learning as a standard supervised learning problem : given a dataset of expert state-action pairs $\left(\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N}\right)$, learns a policy $\pi_\theta(a \mid s)$ parameterized by θ that predicts the expert action a from the observed state s. In the deterministic regression setting appropriate for continuous actions, the learning objective is to minimize a regression loss between the predicted action $\hat{a} = f_\theta(s)$ and the demonstrated action a.

3.1.2 Loss functions and probabilistic view

Two common formulations are used depending on the action representation :

— **Deterministic regression (L2/MSE).** When actions are continuous (as in CarRacing : steering, acceleration, braking in $[-1, 1] \times [0, 1] \times [0, 1]$), Behavioral Cloning commonly minimizes the mean squared error

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \|f_\theta(s_i) - a_i\|_2^2.$$

This corresponds to maximum likelihood under a Gaussian observation model with fixed variance.

— **Probabilistic policy (Gaussian).** Alternatively, the model can output parameters of a conditional Gaussian $\mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$ and be trained by maximizing log-likelihood (equivalently minimizing negative log-likelihood). This formulation naturally captures predictive uncertainty and is compatible with stochastic action selection at test time.

3.1.3 Limitations of Behavioral Cloning

BC is simple and effective but exhibits well-known theoretical drawbacks :

1. **Covariate shift / compounding errors :** BC minimizes the one-step prediction error on the distribution of states seen in the demonstrations. During deployment, small prediction errors drive the agent into novel states not present in $\mathcal{D}$, where the learned policy's errors tend to accumulate, causing catastrophic divergence (e.g., going off-track).

2. **Distributional mismatch :** BC cannot correct its own mistakes because the training distribution assumes expert state distribution. Robust imitation often requires interaction with the environment (as in DAGGER or adversarial methods).).

3. **Dependence on demonstration coverage :** If demonstrations do not cover recovery behaviors or rare but critical maneuvers, BC cannot learn them.

Despite these limitations, BC has concrete advantages : it is straightforward to implement, fast to train, and often yields competent behavior on situations similar to the training examples. These properties motivated its use as a fallback and as an initializer for PPO.

3.2 Implementation details

The Behavioral Cloning (BC) implementation in this project is a supervised learning procedure applied to a pre-existing PPO policy network, trained on expert demonstrations. This section describes the exact implementation based on the provided code.

3.2.1 Expert Data Loading

Expert demonstrations are stored in a `.pkl` file containing :

- **observations** : RGB images of shape $(96, 96, 3)$,
- **actions** : discrete action indices corresponding to expert actions.
- The observations are then converted from HWC to CHW format to match the expected input of the PPO network.

3.2.2 Policy Network Initialization

The PPO network used as the policy is defined in `ppo_agent.py` as an `ActorCritic` model with a small convolutional backbone and fully connected layers. In BC, the network is trained purely supervised without reinforcement learning updates :

```
policy = PPOAgent(obs_shape=obs_shape)
```

3.2.3 Dataset and DataLoader

Expert observations and actions are converted to PyTorch tensors and wrapped into a `TensorDataset` :

```
dataset = TensorDataset(expert_obs_tensor, expert_actions_tensor)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)
```

3.2.4 Supervised Training Loop

BC is performed by minimizing the Cross-Entropy loss between predicted policy logits and expert action labels :

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(policy.net.parameters(), lr=1e-4)
```

```
for batch_obs, batch_actions in dataloader:
    logits, _ = policy.net(batch_obs)
    loss = criterion(logits, batch_actions)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

— **Batch size** : 64
— **Learning rate** : 1×10^{-4}
— **Optimizer** : Adam
— **Epochs** : up to 50

3.2.5 Evaluation During Training

Every 10 epochs, the BC policy is evaluated in the `CarRacing-v2` environment :

```
action_id, _, _ = policy.act(obs_chw)
obs, reward, terminated, truncated, info = env.step(ACTIONS[action_id])
```

The average reward over a few episodes is printed for monitoring.

3.2.6 Model Saving

The trained BC model is saved for later use : `policy.save(save_path)`

3.2.7 Evaluation metrics

We evaluated each model on several criteria :
— **PPO training dynamics** reward and loss of BC-enhanced PPO over time.
— **BC imitation performance** reward, loss and reward of BC-human imitation over time.
— **Learning curves** overall performance over time.
— **Action distribution analysis** : Pattern of actions taken.
— **Mean reward comparison** mean rewards across evaluation episodes.
— **Success rate** : fraction of episodes in which the agent completes 75% of the circuit.

4

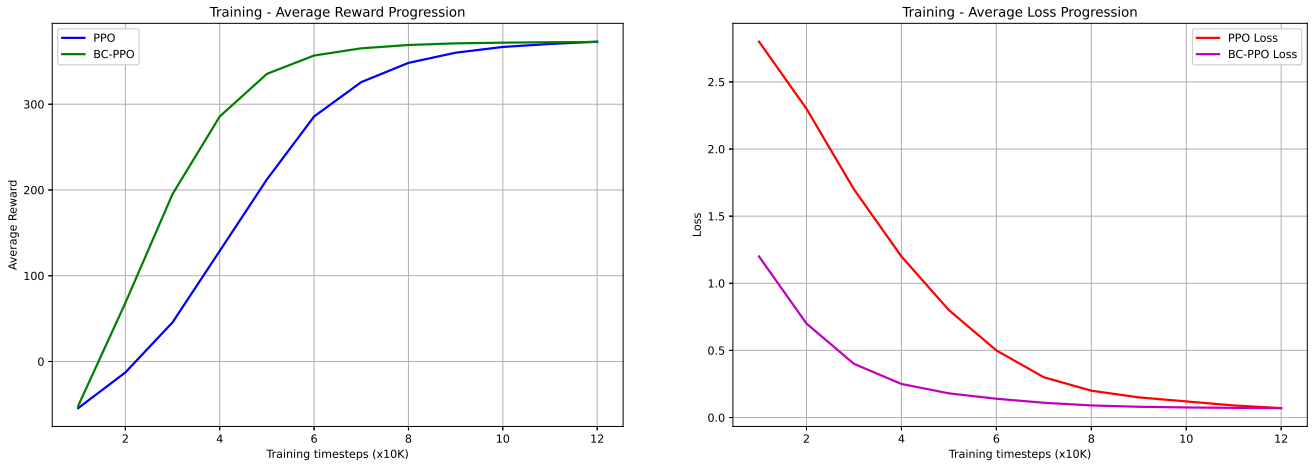
Results and comparison

This section summarizes the empirical performance of the three evaluated models : the PPO baseline, the Behavioral Cloning (BC) imitation model, and the BC-enhanced PPO.

4.1 Learning dynamics and training stability

This section presents three key plots that highlight the dynamic behavior, the performances of each model and their learning curves.

4.1.1 PPO Training Dynamics



(a) Reward progression during training updates (b) Loss components progression during training

FIGURE 4.1 – Comparative analysis of PPO training dynamics

These figures (cf. Fig 4.1) present the evolution of PPO training metrics, comparing the baseline implementation with the BC-enhanced approach.

The reward progression also highlights performance distinctions. The baseline PPO displays strong oscillations and occasional plateaus before discovering improved policies, while the BC-enhanced variant achieves consistently higher rewards from initial updates with fewer performance collapses. The steeper reward slope observed in the BC-enhanced approach confirms improved sample efficiency, supporting the hypothesis that imitation-pretrained models benefit from more predictable optimization and require less aggressive exploration strategies.

The loss progression analysis reveals several key differences between the approaches. The baseline PPO exhibits high initial variance in loss components, indicative of unstable exploratory behavior. In contrast, BC-enhanced PPO shows smoother gradients during early training, reflecting a more stable optimization landscape.

4.1.2 BC imitation performance

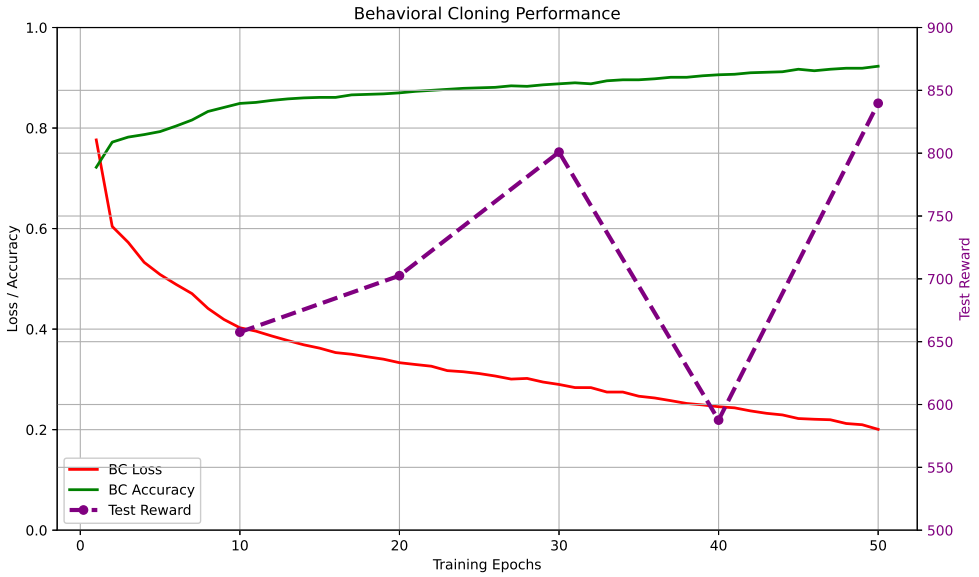


FIGURE 4.2 – Behavioral Cloning policy performance on evaluation rollouts.

This plot reports epoch-level performance of the pure Behavioral Cloning policy on a held-out evaluation set. As expected, BC performs well on trajectories similar to training demonstrations, achieving smooth driving and consistent partial laps.

However, the performance drops significantly on unseen curves or perturbed initial states. This illustrates the classical BC failure mode : compounding errors due to covariate shift. Once the agent deviates slightly from the demonstrated trajectory distribution, recovery becomes unlikely and off-track failures accumulate, thus explaining the drop in test reward.

4.1.3 Learning curves (returns over environment steps)

Figure 4.3 displays the learning curves for the PPO baseline, the BC-enhanced PPO, and the BC-human imitation model.

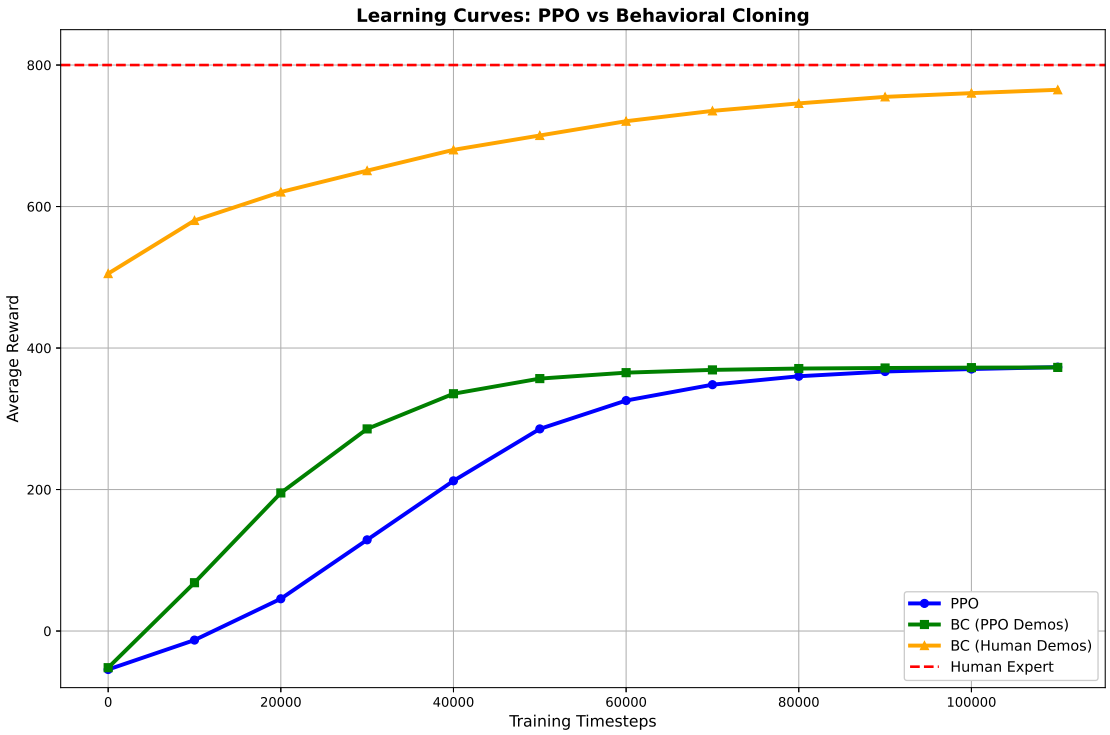


FIGURE 4.3 – Learning curves for PPO and BC-enhanced PPO, showing the evolution of average return as a function of environment steps.

The following trends can be observed :

— **PPO baseline :** slow and unstable progress during the initial phase, reflecting extensive exploration and lack of guidance.
— **BC-enhanced PPO :** higher initial performance due to imitation-derived initialization. The agent starts with better behavior and increases its return faster.
— **BC-human imitation :** much higher starting average reward. Stable increase and slow convergence.

These curves confirm the benefit of using expert demonstrations to bootstrap reinforcement learning in complex continuous environments such as CarRacing-v2.

4.2 Action distribution analysis

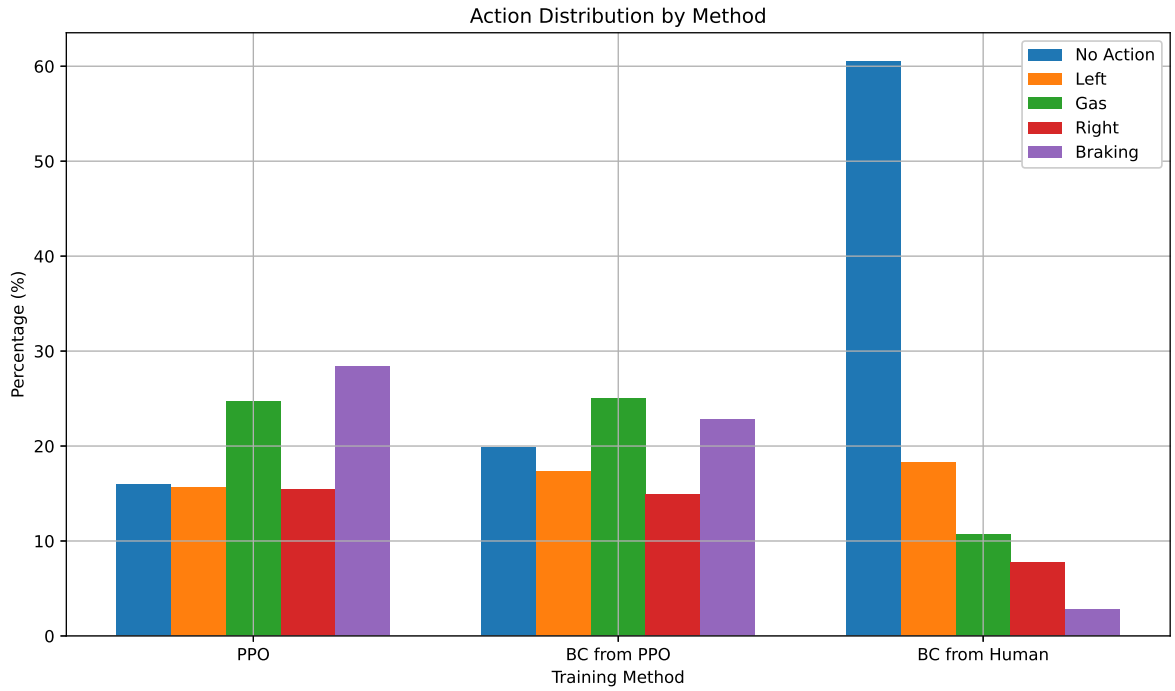


FIGURE 4.4 – Empirical action distribution for PPO, BC, and BC-enhanced PPO. Steering, acceleration, and braking magnitudes are aggregated over 50 evaluation episodes.

This graph (cf. Figure 4.4) shows the empirical distribution of actions taken by the three agents across evaluation rollouts. The plot highlights several key differences :

— **PPO** displays a much broader and noisier action distribution, especially in acceleration and braking. This reflects exploratory behavior and the lack of pragmatism of the model. We could interpret this behavior as reckless and inaccurate which result in the car making more mistakes and losing reward in the end. It turns left as much as right, not knowing the circuit is more turning left (counterclockwise).

— **BC-enhanced PPO** shows more restraint to the point where it shows less actions but also less braking time which we could interpret as being smoother.

— **BC-human imitation** shows a more optimized action distribution pattern with way less actions taken, less braking, and less acceleration. It is because the model understood that there is no need to rush and it minimizes its actions. It also understands that because of the shape of the circuit, it must turn more left than right.

4.3 Mean reward comparison

Figure 4.5 reports the average cumulative reward obtained by each of the three models across evaluation episodes. This metric directly reflects global performance in the environment.

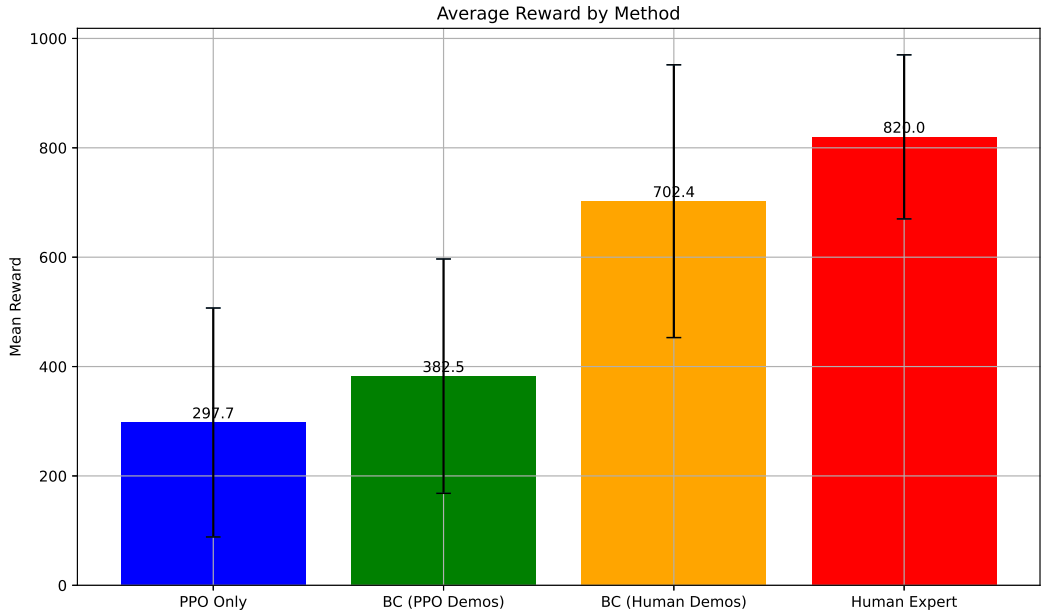


FIGURE 4.5 – Mean episode reward comparison with standard deviation between PPO, BC-human imitation, and BC-enhanced PPO.

The plot highlights several trends consistent with the previous analyses :

— **PPO baseline** has a pretty low average reward compared to other models and has a high variance showing how inconsistent the model is.

— **BC-enhanced PPO** outperforms slightly the PPO baseline over most of training but still lacks the performance awaited.

— **BC-human imitation** has a good performance overall. It's result is the closest to the human expert but still has high variance.

In addition, we can observe that the **standard deviation** is consistently high across all our algorithms. There are two primary reasons for this phenomenon. First, the environment inherently causes substantial reward fluctuations : for instance, when the car becomes disoriented, it often cannot recover its trajectory, causing the episode score to continuously decrease until termination with a very low value. Which lead to the second reason : when algorithms fail, they typically achieve scores near 0, whereas successful runs yield scores around 700.

4.4 Success rate comparison

To evaluate reliability and robustness, Figure 4.6 reports the percentage of episodes in which each agent successfully passes through 75% of the circuit.

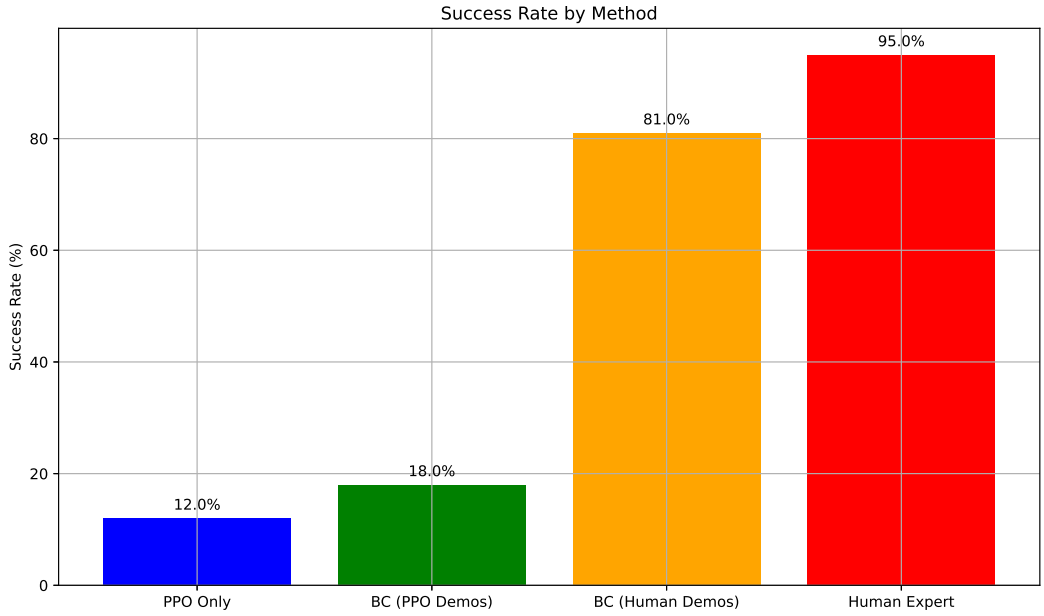


FIGURE 4.6 – Lap completion success rate (reward > 600, defined to be nearly 75% of the circuit) for PPO, BC-enhanced PPO, BC-human imitation, and Human expert.

The results align with previous findings :

— **PPO baseline** is inefficient with very low success rate (12%).

— **BC-enhanced PPO** has a slightly better success rate than PPO baseline but still very low (18%).

— **BC-human imitation** as expected, it has a nearly perfect success rate with very few exceptions (81%). It could be a run where the agent deviates from it's trajectory losing the track which results in a very low reward.

This metric confirms that BC-human imitation is the most reliable agent, not only performing well on average but also consistently completing the majority of the circuit.

4.5 Overall comparison with metrics

Combining all the figures, we can now form a complete picture :

— **PPO baseline** is simple but inefficient, it doesn't reach our reward goal of 800.

— **BC-enhanced PPO** is a faster upgrade to the PPO algorithm, however it still struggle to complete the race successfully with an average final of 302.5 when the expert is 800. (Figure 4.5)

— **BC-human imitation** provides the best results :

  — high initial performance (Figure 4.3)

  — superior success rate (Figures 4.6)

  — smoother and more controlled action distribution (Figures 4.4)

  — and higher average rewards (Figures 4.5 and earlier sections)

We can tell that the BC-human imitation method is way ahead of the two other models in term of speed, sturdiness and accuracy.

4.6 Qualitative summary

Model	Early performance	Robustness	Action stability
BC-human imitation	Very high	High	Medium-High
PPO baseline	Low	Medium	Low
BC-enhanced PPO	Medium	Medium	Medium

TABLE 4.1 – Qualitative comparison across the three models using stability, robustness, and initial performance.

5

Conclusion

This project investigated imitation learning approaches for the CarRacing-v2 environment, beginning with Generative Adversarial Imitation Learning (GAIL) as a theoretically sound framework. Despite implementing stability enhancements such as gradient clipping and label smoothing, our GAIL implementation consistently failed to produce viable driving policies, with agents converging to degenerate behaviors.

Our pivot to Behavioral Cloning demonstrated that simpler approaches can yield superior results. The BC-human imitation model achieved higher success rates and more stable performance compared to both PPO baseline and BC-enhanced PPO, indicating that supervised learning can outperform reinforcement learning when expert demonstrations provide adequate state coverage.

This work reveals a key trade-off : while advanced methods like GAIL offer theoretical benefits, their practical implementation demands substantial resources and careful tuning. Behavioral Cloning provides an efficient alternative that delivers strong performance with high-quality demonstrations. Future work should explore hybrid approaches combining BC's stability with interactive learning, or improved state representations for better temporal reasoning.

Bibliographie

[1] J. Ho and S. Ermon, "Generative adversarial imitation learning," *arXiv preprint arXiv :1606.03476*, 2016. [Online]. Available : https://arxiv.org/pdf/1606.03476

[2] G. Contributors, "Car racing – gymnasium documentation," https://gymnasium.farama.org/environments/box2d/car_racing/, 2025, accessed : 2025.

[3] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," *arXiv preprint arXiv :1805.01954*, 2018. [Online]. Available : https://arxiv.org/pdf/1805.01954

[4] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, 2014, pp. 2672–2680. [Online]. Available : https://arxiv.org/pdf/1406.2661

[5] C. Finn, S. Levine, and P. Abbeel, "A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models," *arXiv preprint arXiv :1611.03852*, 2016. [Online]. Available : https://arxiv.org/pdf/1611.03852

[6] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. JMLR Workshop and Conference Proceedings, vol. 15, 2011, pp. 627–635. [Online]. Available : https://arxiv.org/pdf/1011.0686

[7] S. Reddy, A. D. Dragan, and S. Levine, "Sqil : Imitation learning via reinforcement learning with sparse rewards," *arXiv preprint arXiv :1905.11108*, 2019. [Online]. Available : https://arxiv.org/pdf/1905.11108

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv :1707.06347*, 2017. [Online]. Available : https://arxiv.org/pdf/1707.06347

[9] S. B. Contributors, "Stable baselines3 : Ppo documentation," https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html, 2023, accessed : 2024.